

CLDV POE Report

1. Introduction

This project forms part of the Cloud Development (CLDV) Practical Outcomes Evaluation (POE) and focuses on developing a cloud-based web application that demonstrates the use of Microsoft Azure services in a real-world scenario. The objective of the system is to implement a venue and event booking solution with support for product and customer management. The project highlights the integration of cloud services such as Azure Table Storage, Blob Storage, Queue Storage, and File Share to store and process data efficiently.

The main objectives are:

- To design and implement a web application using ASP.NET Core MVC that supports full CRUD functionality for both products and customers.
- To integrate cloud storage services into the solution, ensuring persistence, scalability, and reliability.
- To deploy the application to Microsoft Azure App Service, demonstrating cloud deployment and hosting.
- To showcase best practices in structured coding, validation, and user-friendly interface design.

The technologies used in this project include **ASP.NET Core MVC** for application development, **Entity Framework Core** for data access abstraction, and multiple **Azure Storage services** (Table Storage, Blob Storage, Queue Storage, and File Share) for data management. The solution is developed and tested within **Visual Studio 2022**, with hosting and deployment managed via **Azure App Service** and **Azure SQL Database**.

2. System Setup and Configuration

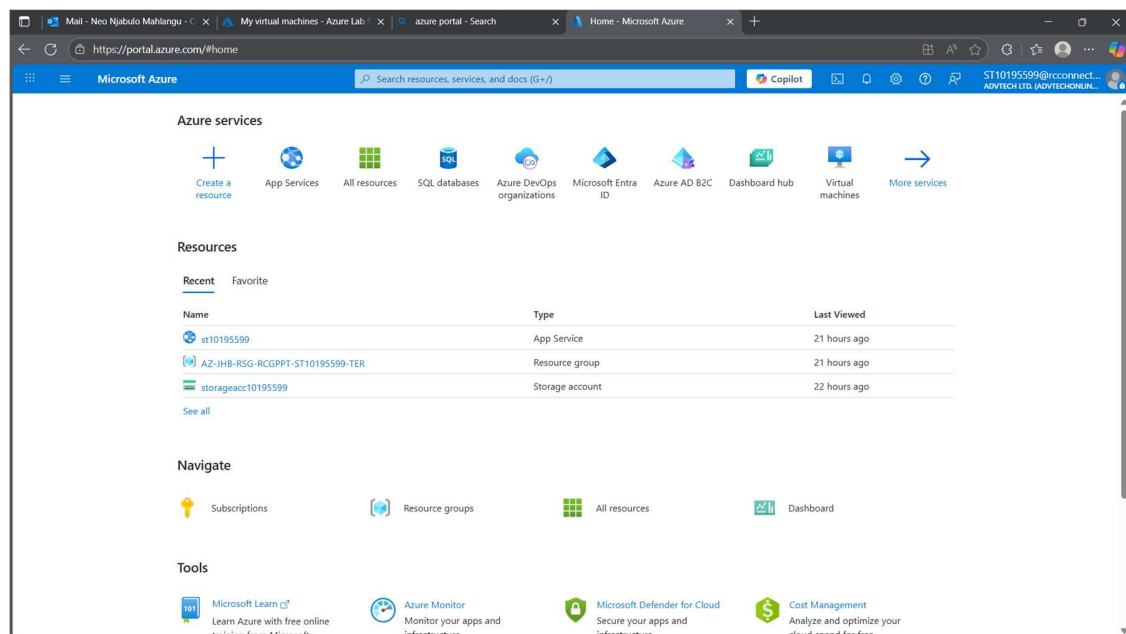
The development environment was set up using **Visual Studio 2022 Community Edition** with the **ASP.NET and web development workload** installed. This provided the necessary tools to build an ASP.NET Core MVC application, including Razor pages, NuGet package support, and integrated debugging. A new MVC project was created and structured into **Models, Views, Controllers**, and **Services** to align with best practices.

On the cloud side, the project made use of **Microsoft Azure**. A dedicated **Resource Group** was created to contain all resources for the solution. Inside this resource group, an **Azure Storage Account** was provisioned. From this single account, multiple storage services were configured to meet different requirements of the project:

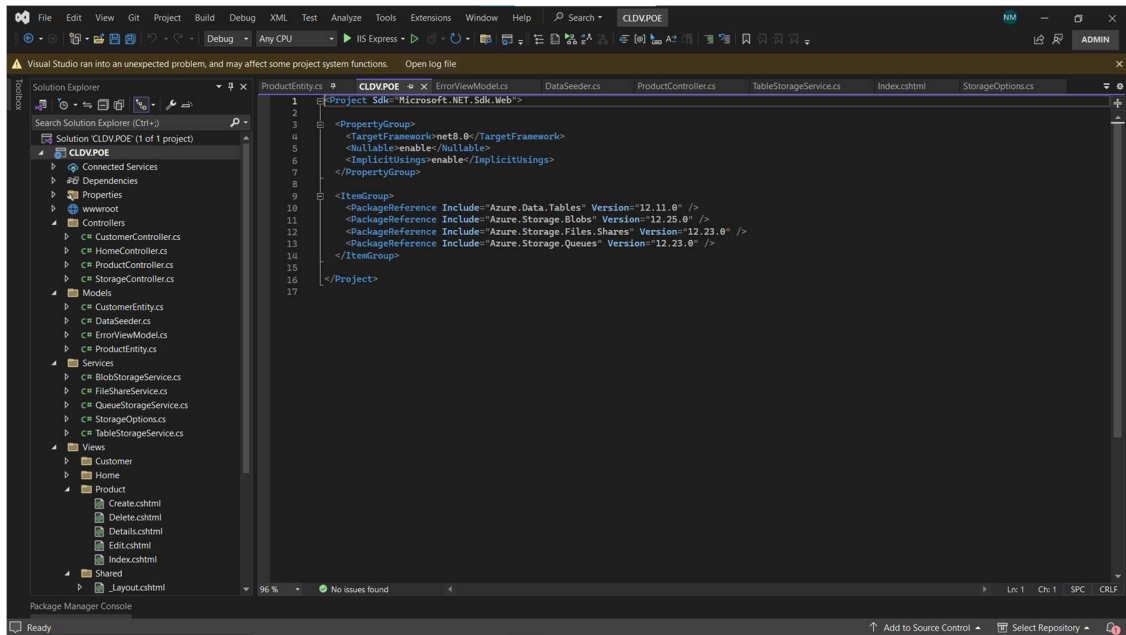
- **Azure Table Storage** – used to persist structured customer and product data.
- **Azure Blob Storage** – configured to store and retrieve product images and other binary files.
- **Azure Queue Storage** – used for background messaging and logging events such as product creation or deletion.
- **Azure File Share** – provided a shared file system to maintain application logs.

Finally, an **Azure App Service** was created to host the application. The publish profile was downloaded from Azure and imported into Visual Studio, enabling a direct deployment pipeline from the development environment to the cloud.

This setup ensured a smooth workflow, where code was developed and tested locally in Visual Studio before being integrated with Azure’s cloud storage services and deployed to the hosted environment.



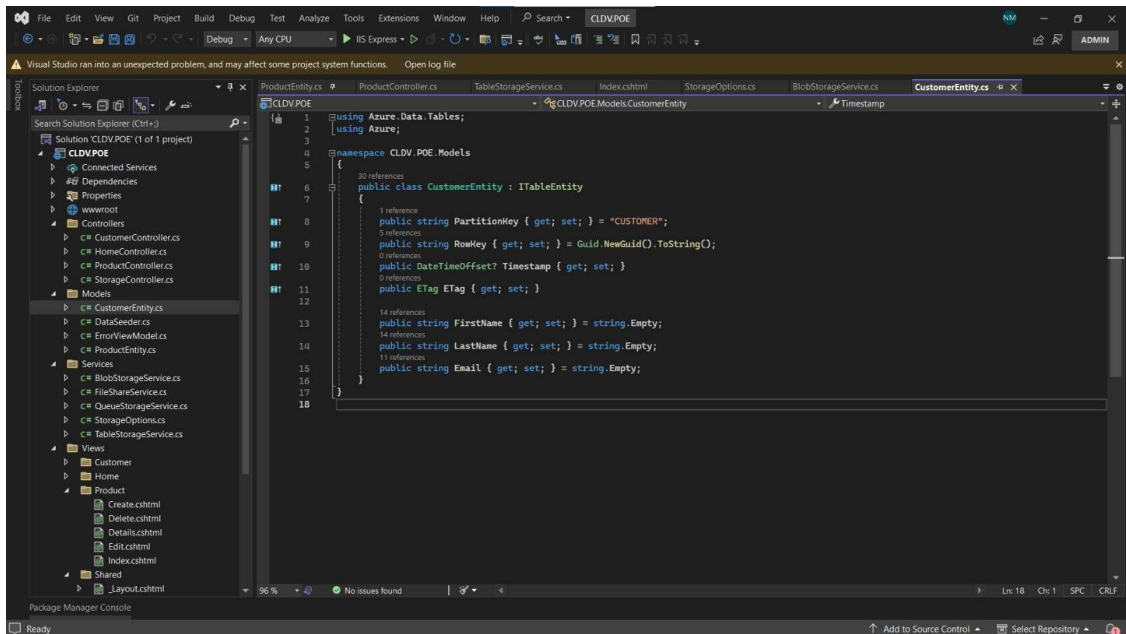
This Is A Screenshot Of My Azure Portal.



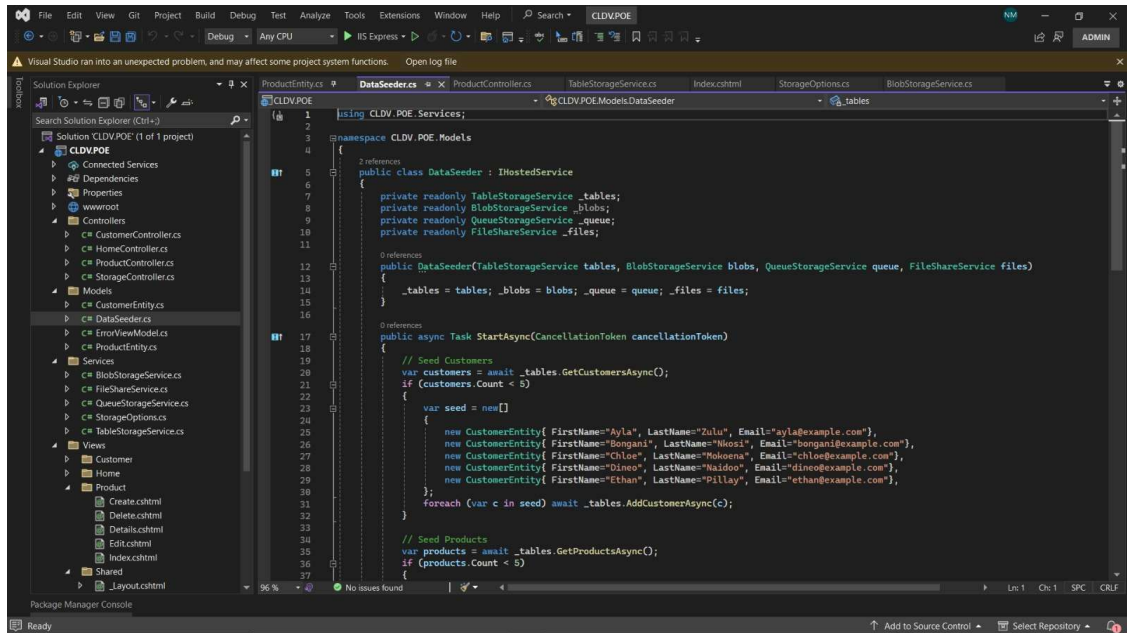
This Is The SetUp On My Visual Studio. As Shown In The ScreenShot I Am Running It On Asp8.0 Which Is One Of The Requirements For My Web App To Be Deployed.

3. Implementation

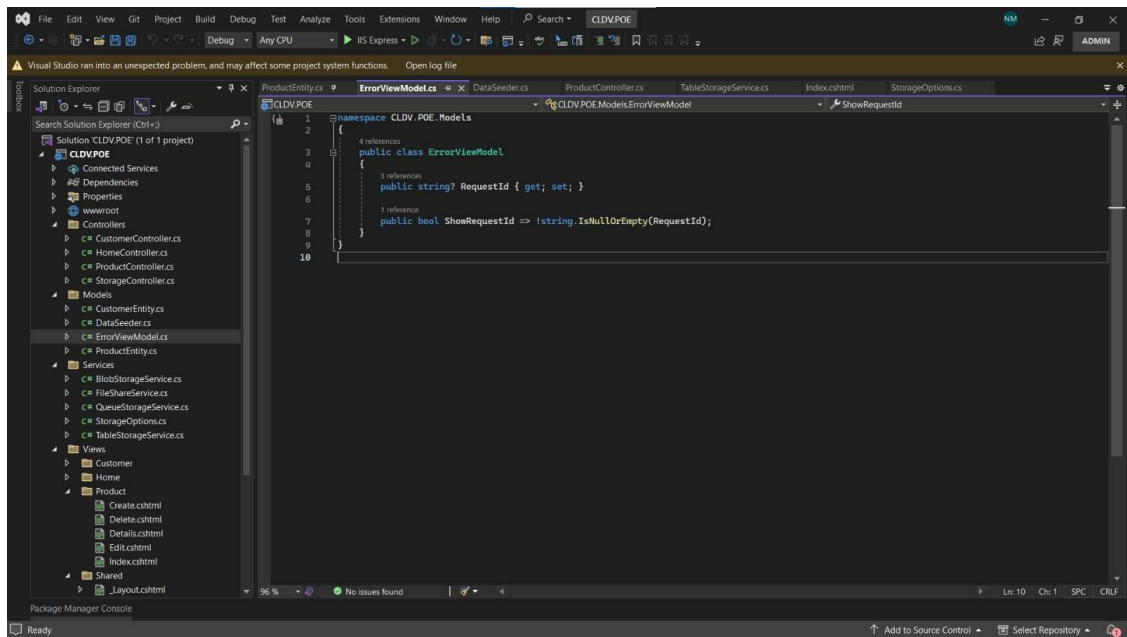
3.1 Models

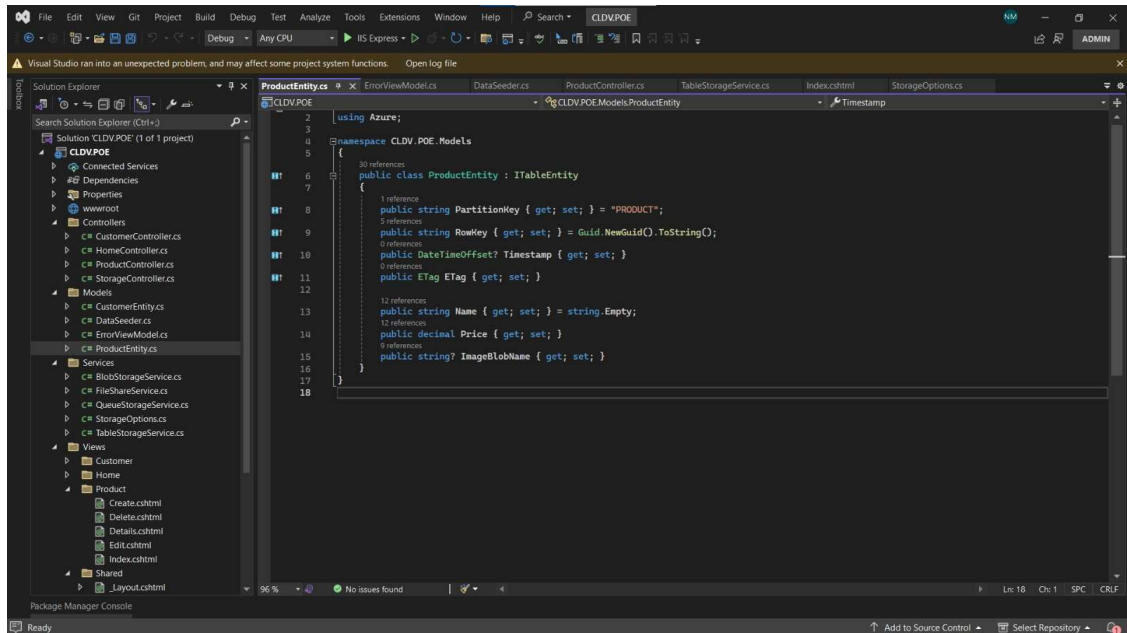


This Is The Code For My Customers In The Models. It Gets And Sets Their Name, Surname, And Email.



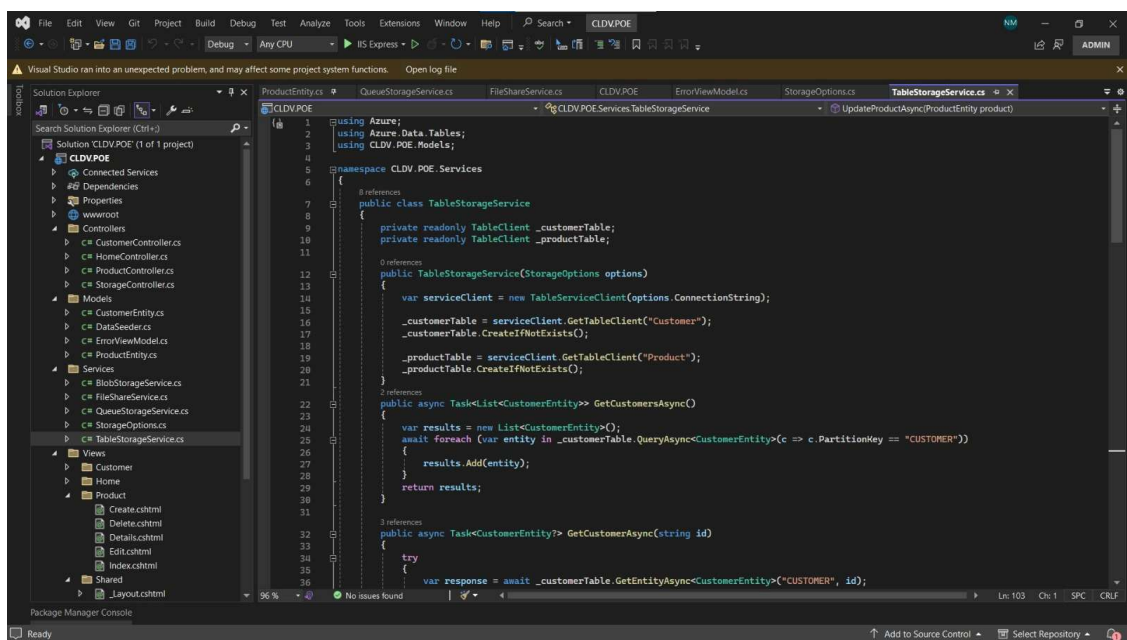
This Is For The DataSeeder Where I Test If The Customers Can Be Stored.



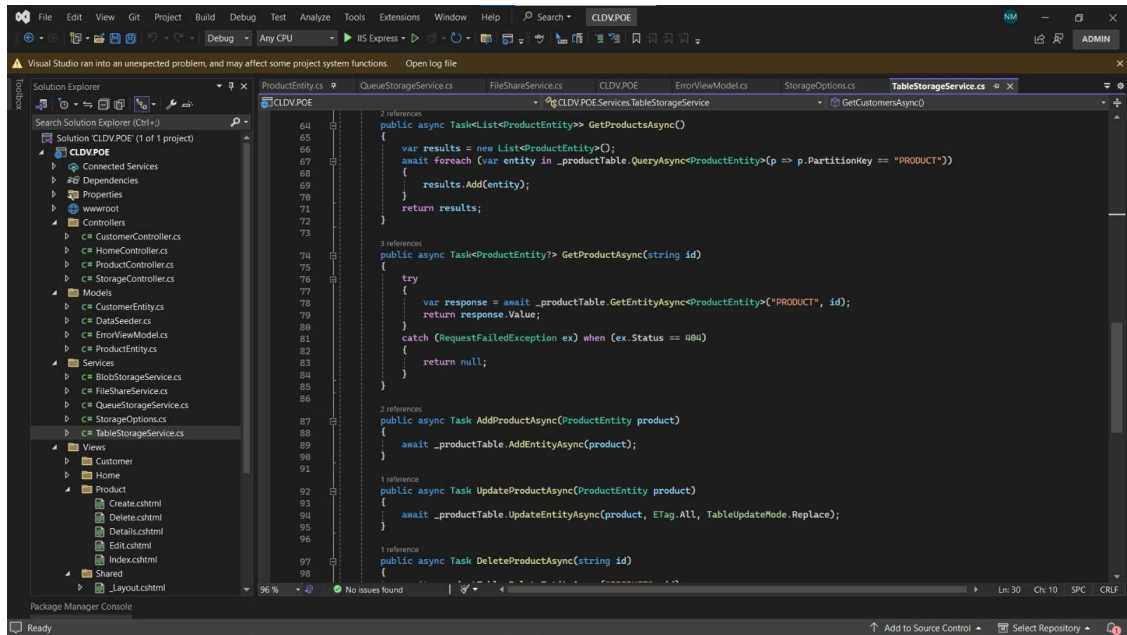


Lastly, This Is For The Product Entity Class Where The Name Of The Product, The Price And The Image Can Be Uploaded Is Stored.

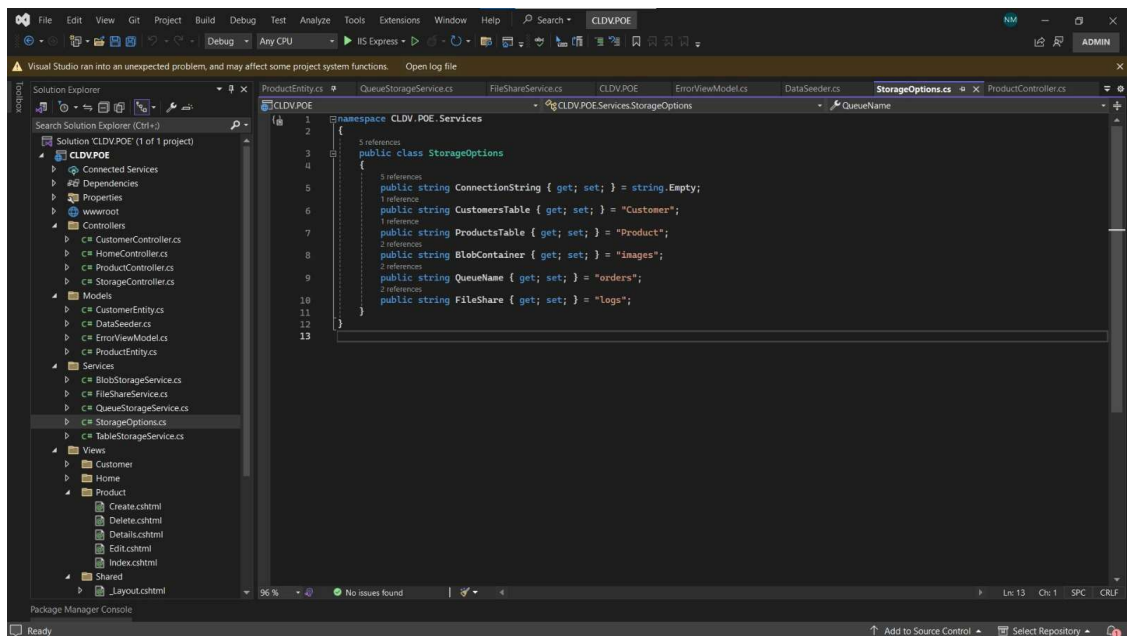
3.2 Services



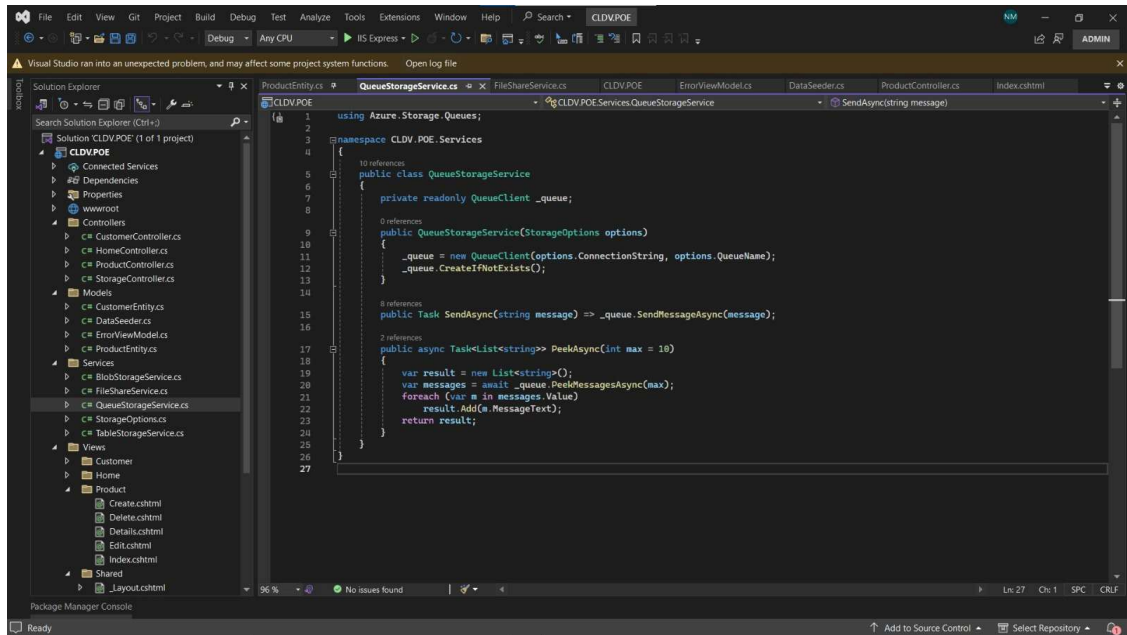
This Is The Table Storage Services, Where The Information Of The Customer Table Is Stored.



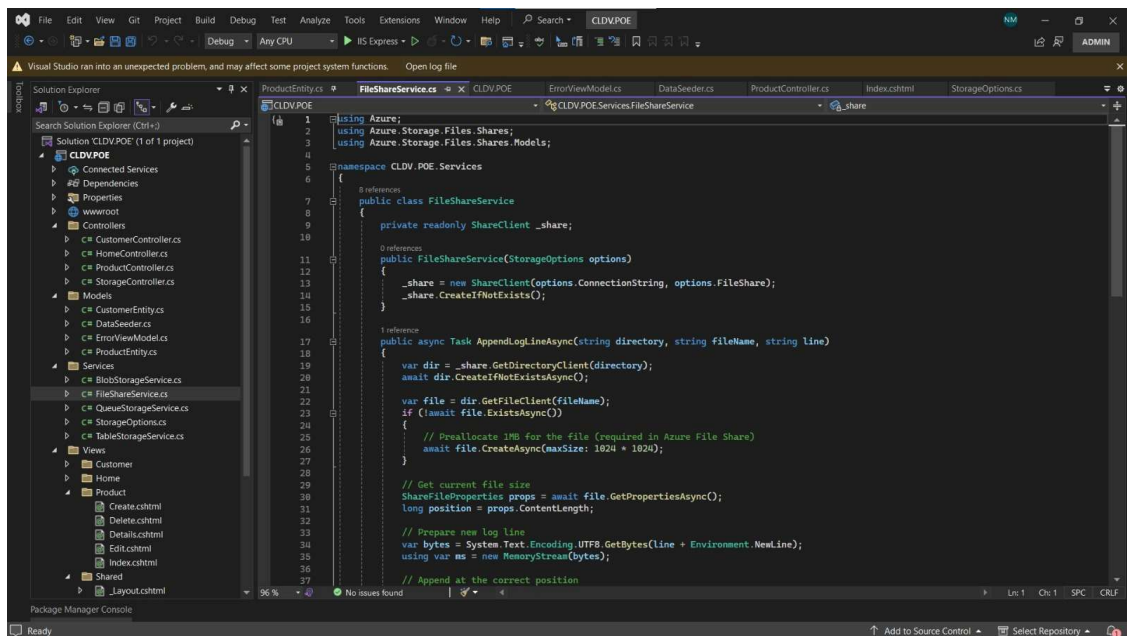
This Is The Same Code But For The Product Table

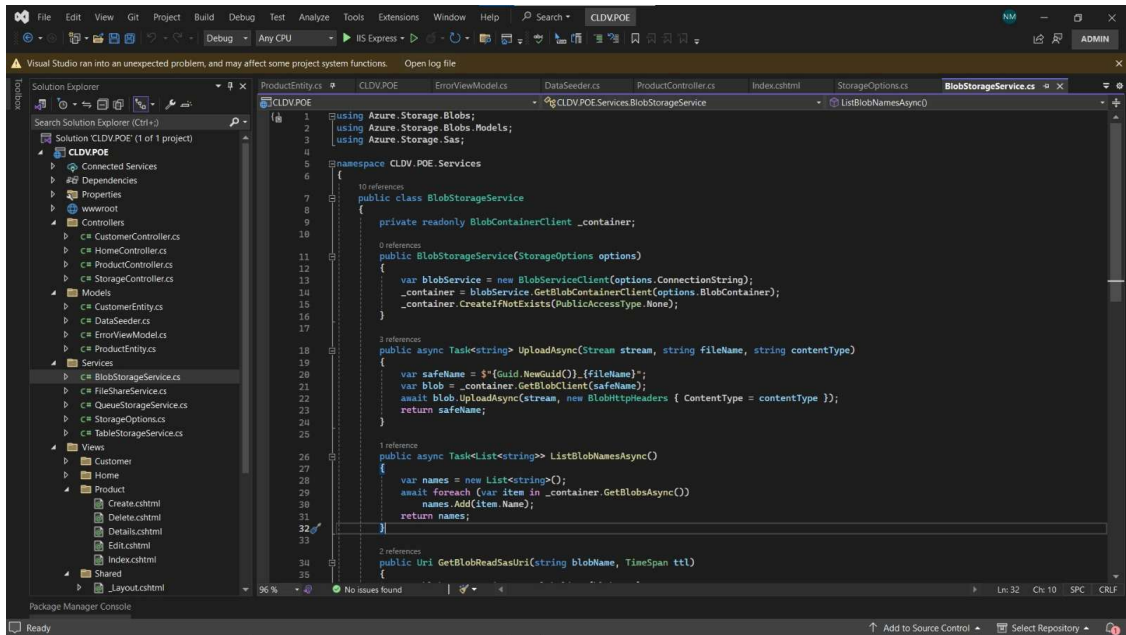


This Is The Storage Options Services Where The Classes For Each Service Is AppSettings Are Declared.



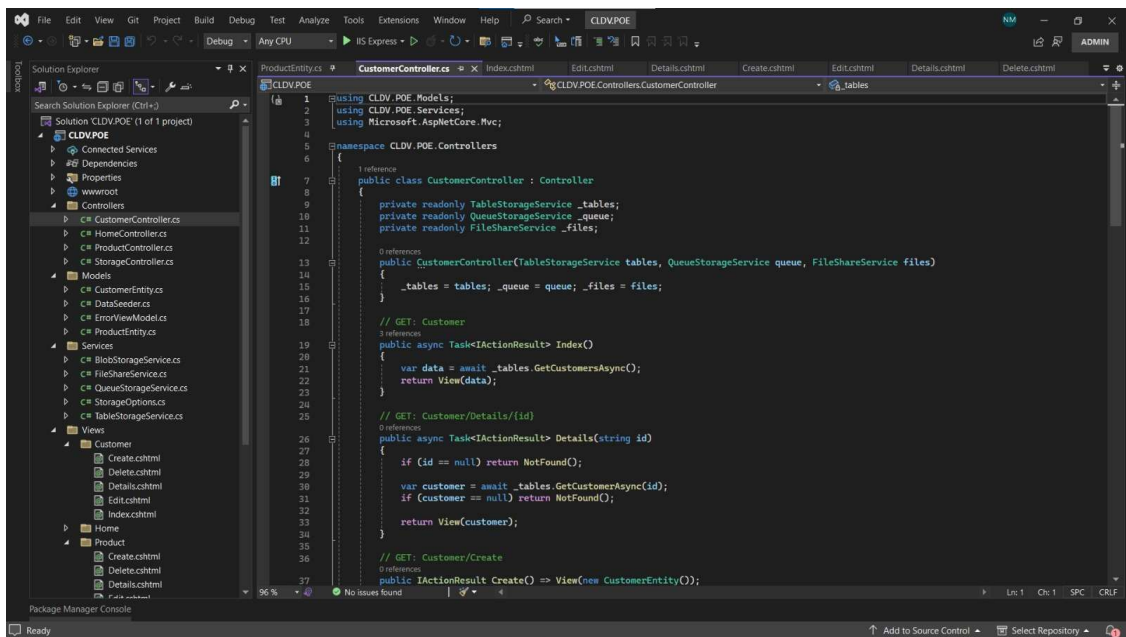
This Is The Queries Services Where It Keeps Track Of Every Action Done In The Web App.



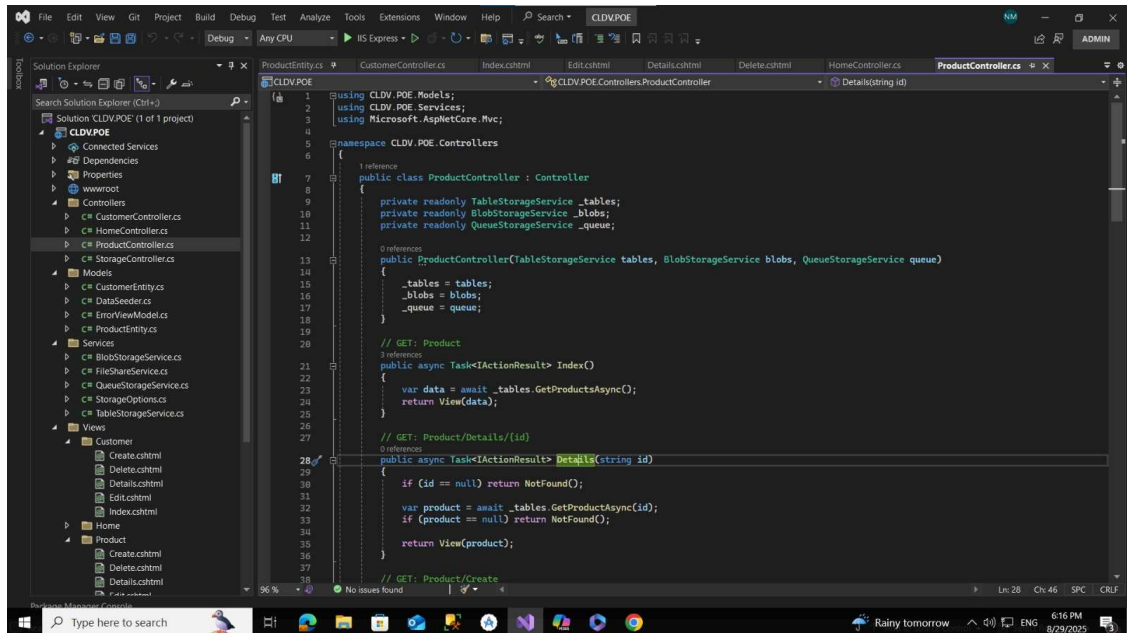


Lastly, This Is The Blob Storage Where The Images Uploaded Can Be Seen.

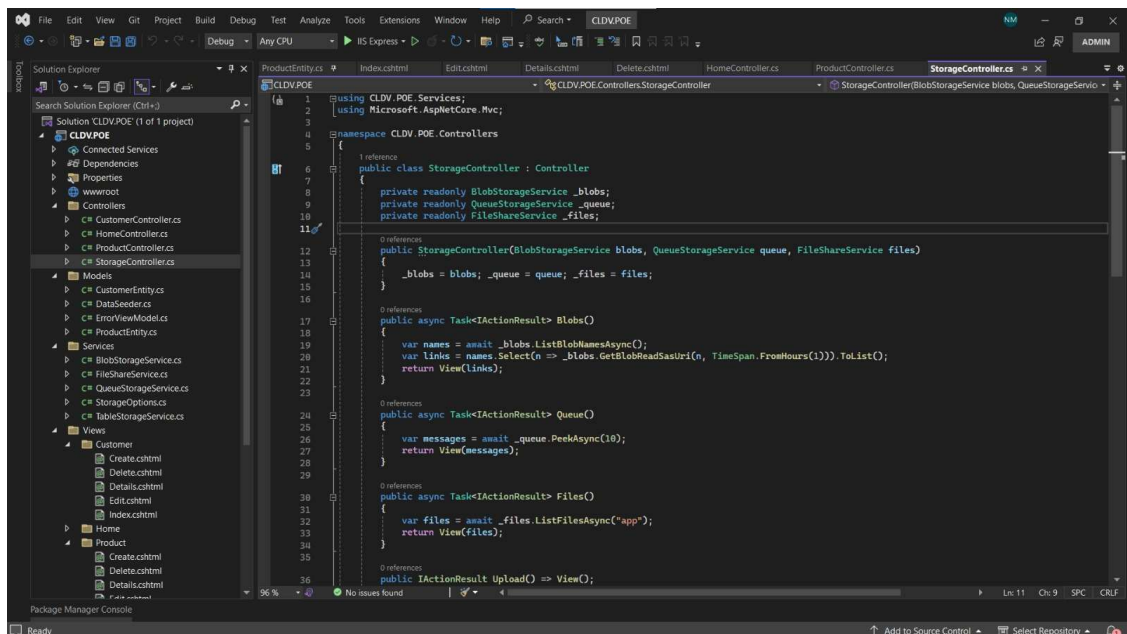
3.3 Controllers



This Is The Customers Controller Where The Create, Details, Delete, Edit, And Index Views Are Created To Configure Each Action.

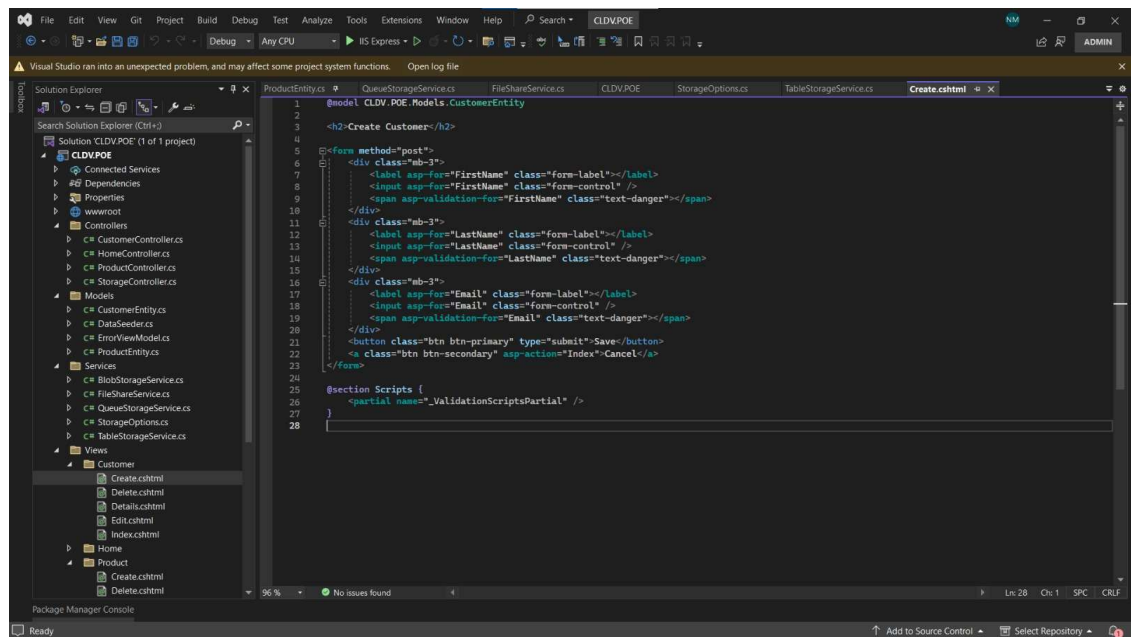


This Is The Product Controller Where The Create, Details, Delete, Edit, And Index Views Are Created To Configure Each Action.

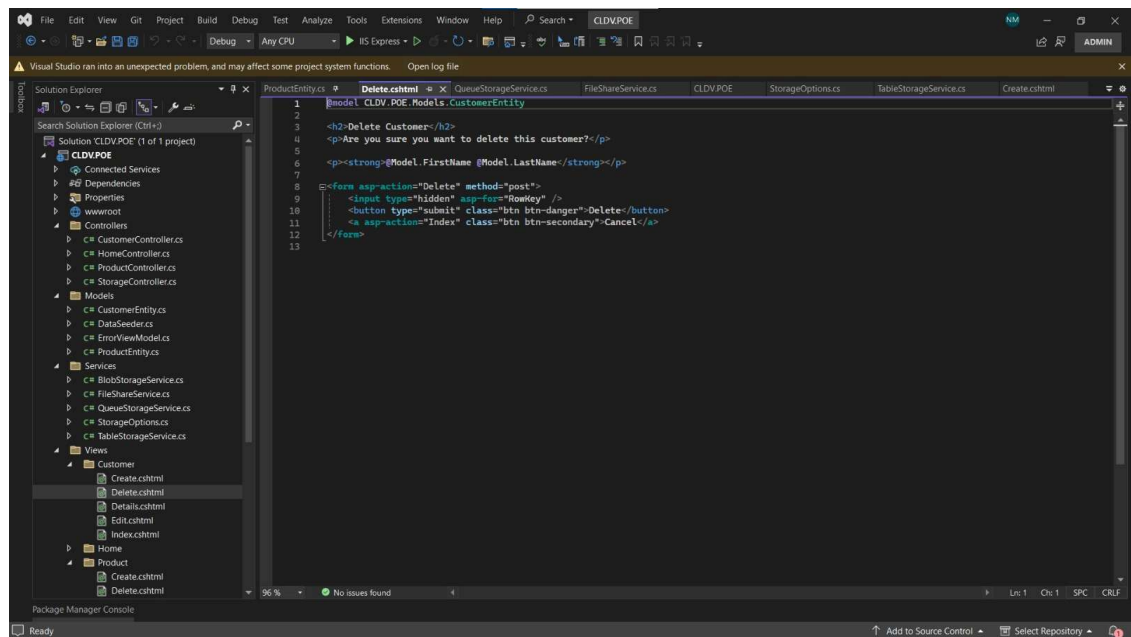


This Is For The Blob, Queries And Files To Be Viewed On The Web Application.

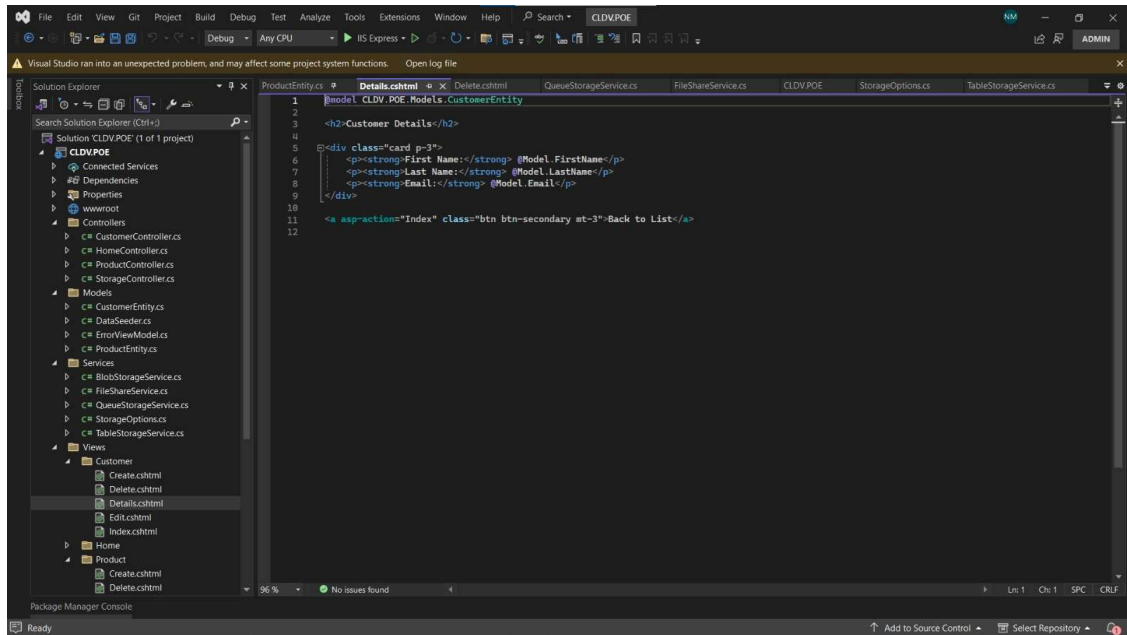
3.4 Views



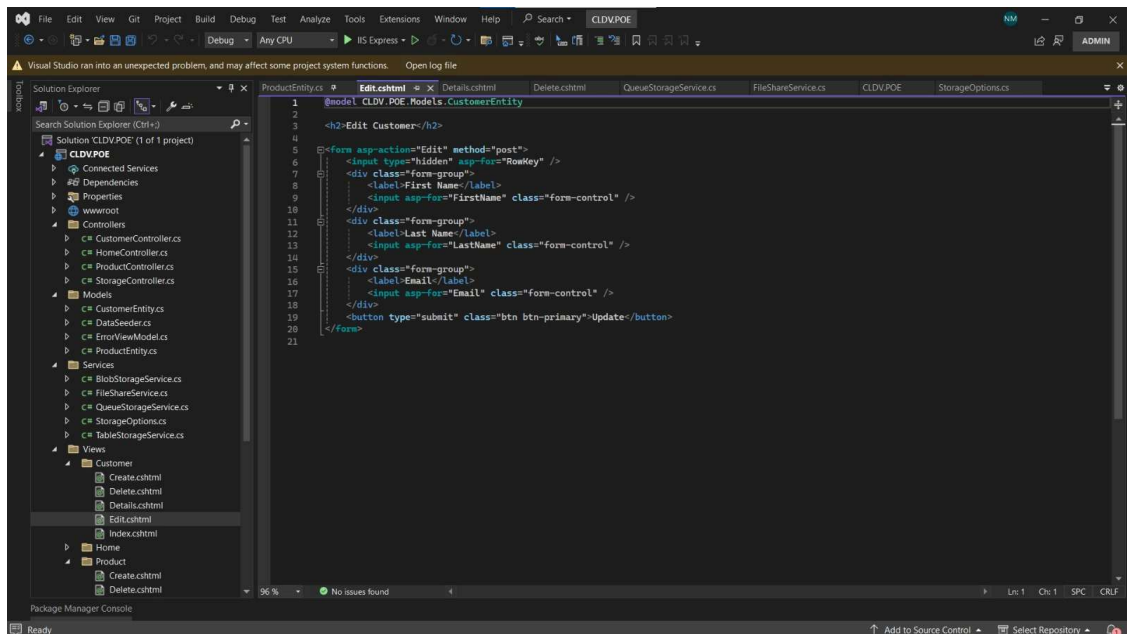
This Is The Create View For The Customers



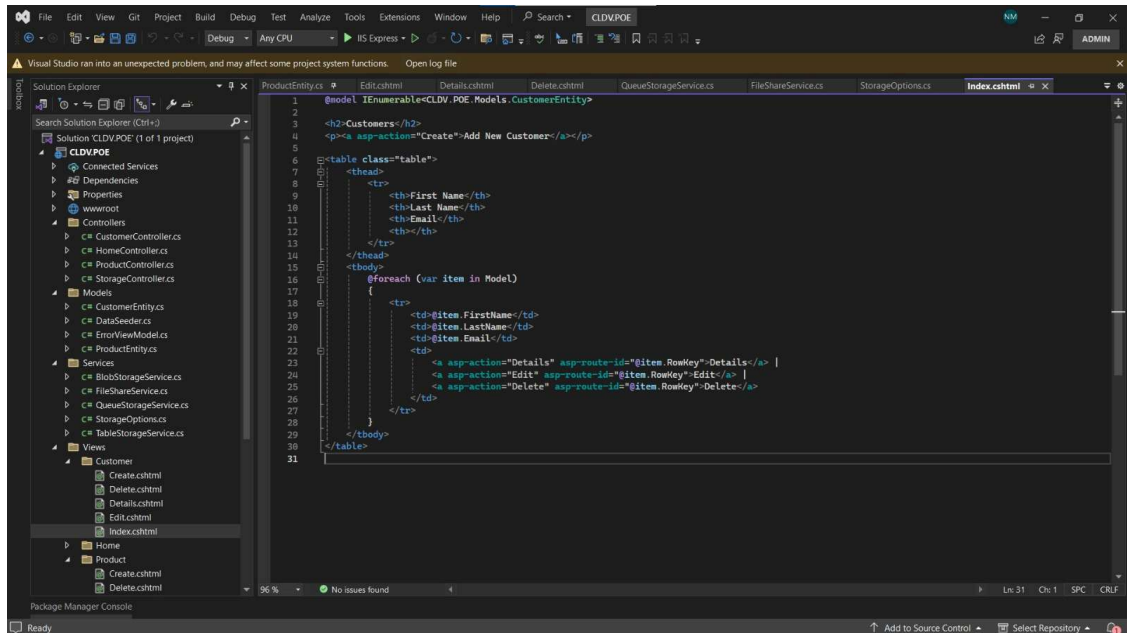
This Is The Delete View For The Customers



This Is The Details View For The Customers

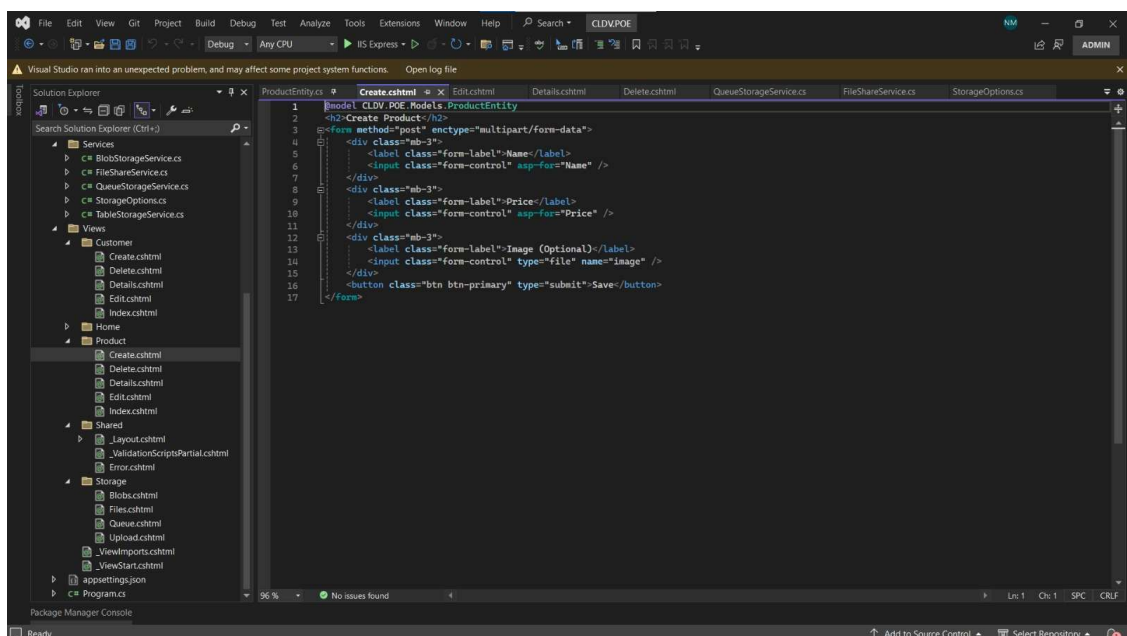


This Is The Edit View For The Customers

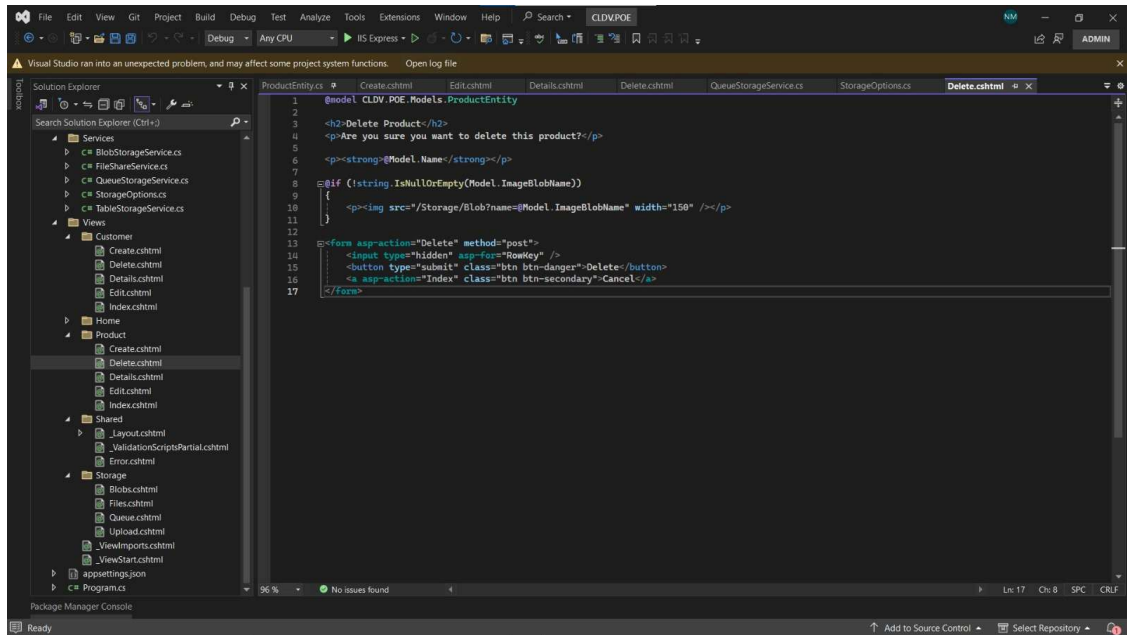


This Is The Index View For The Customers

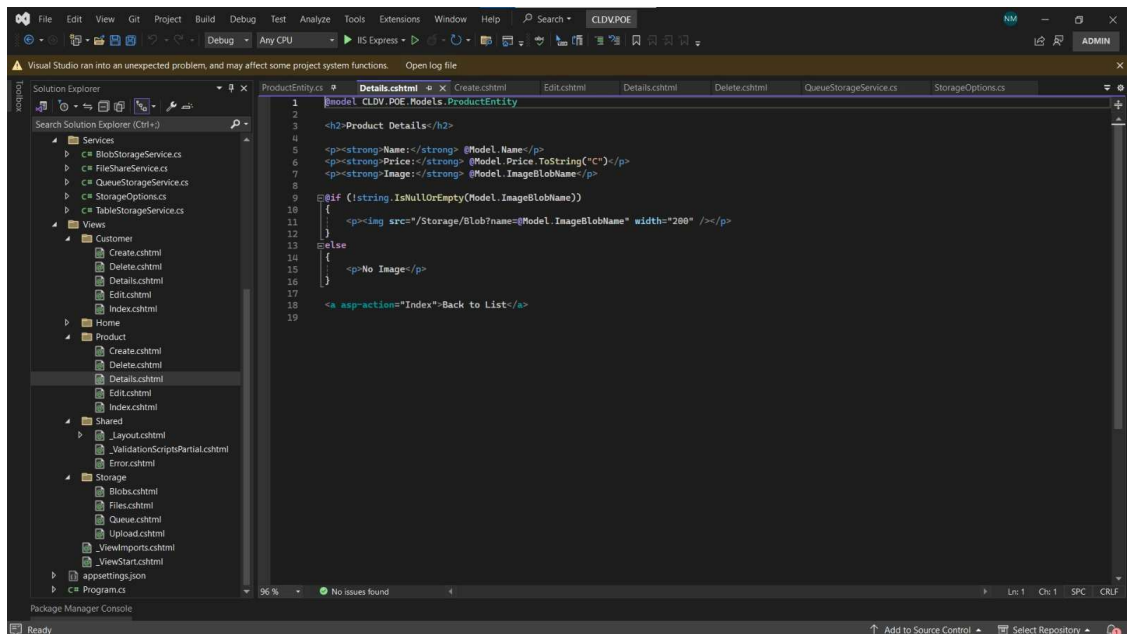
PRODUCT



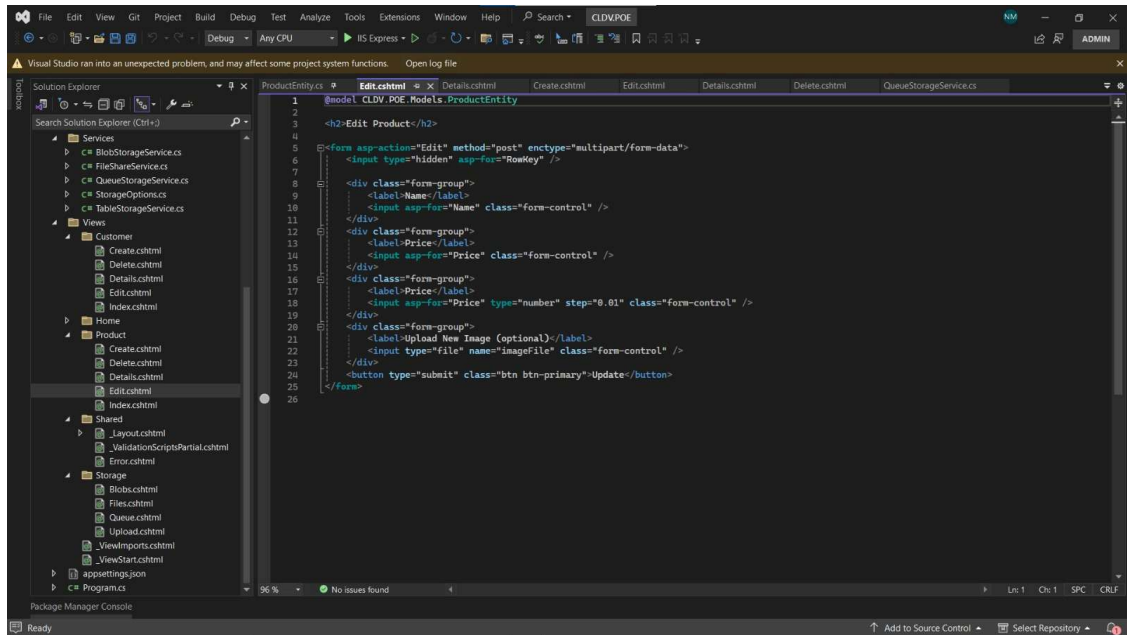
This Is The Create View For The Products



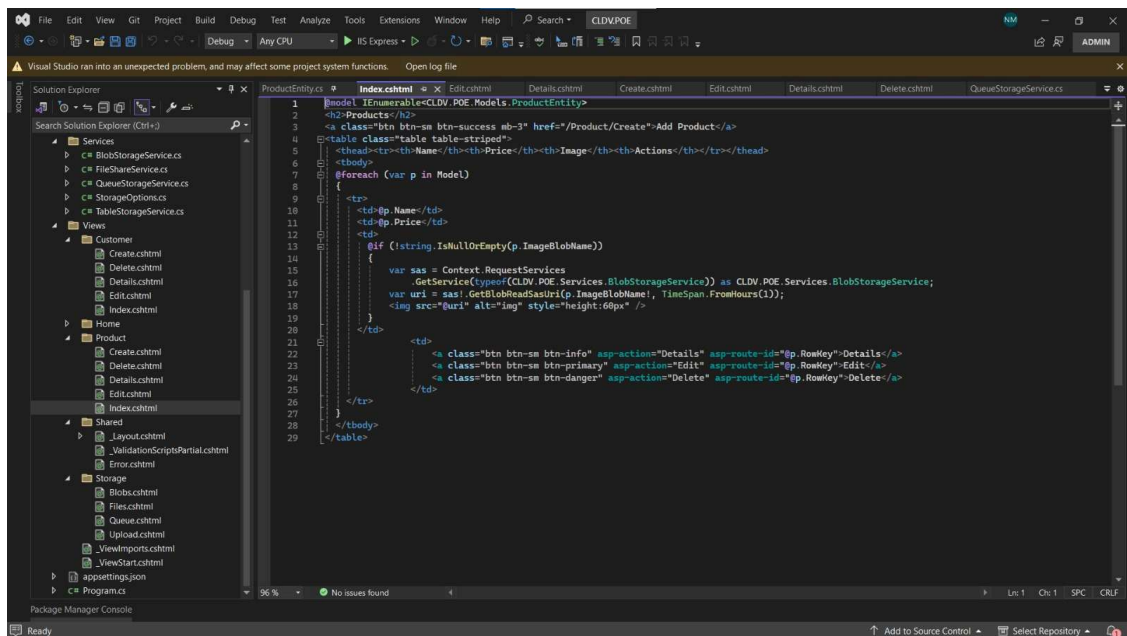
This Is The Delete View For The Products



This Is The Details View For The Products

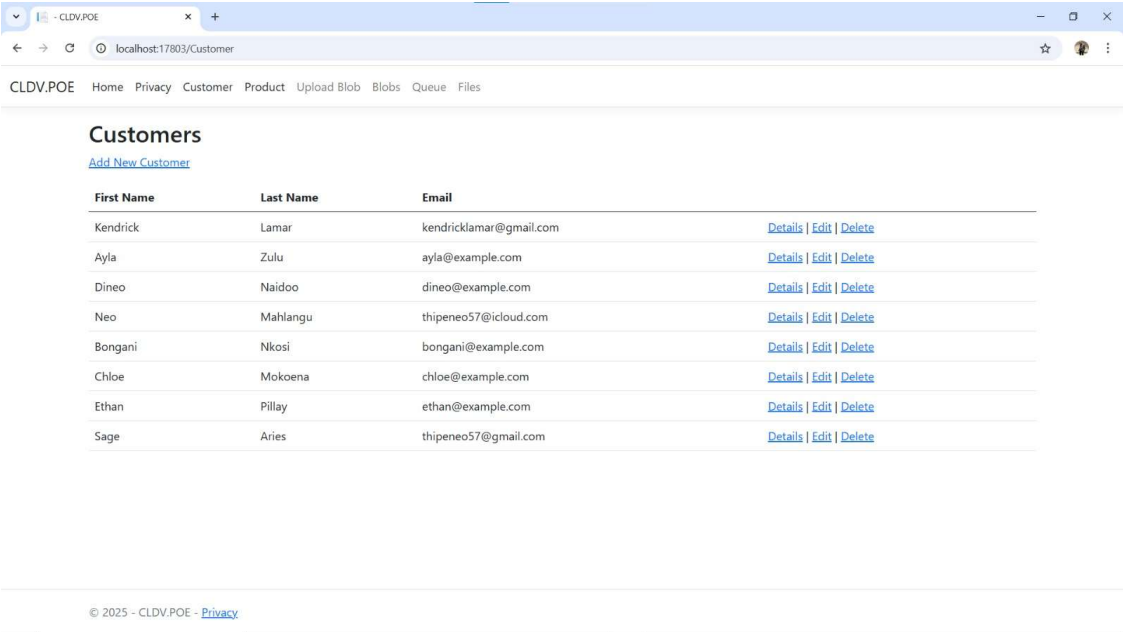


This Is The Edit View For The Products

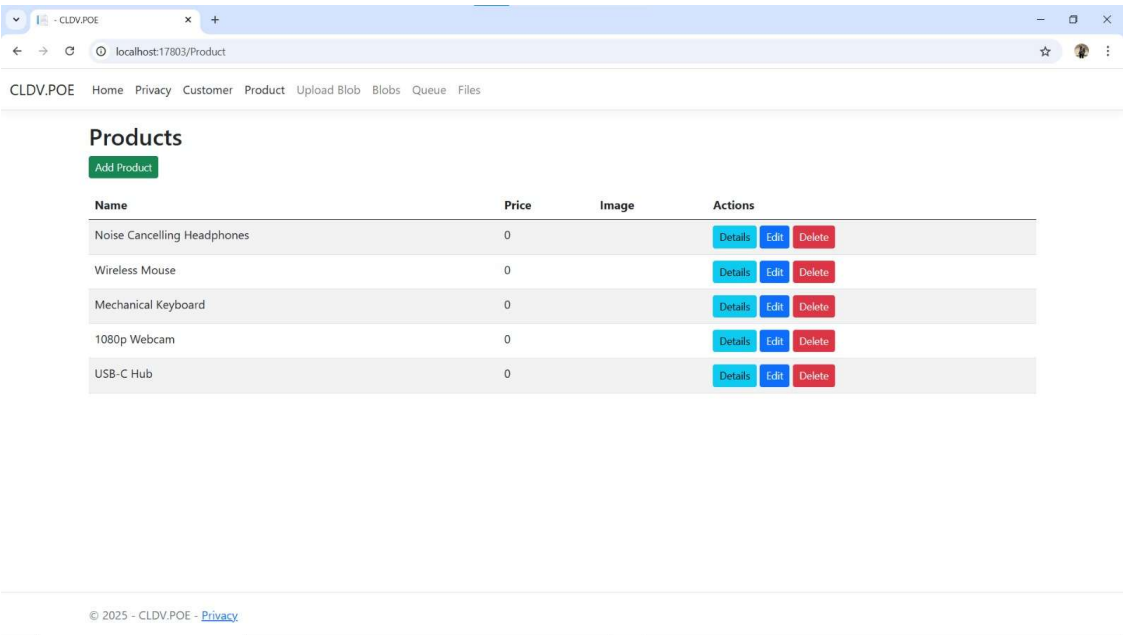


This Is The Index View For The Products

4. Testing



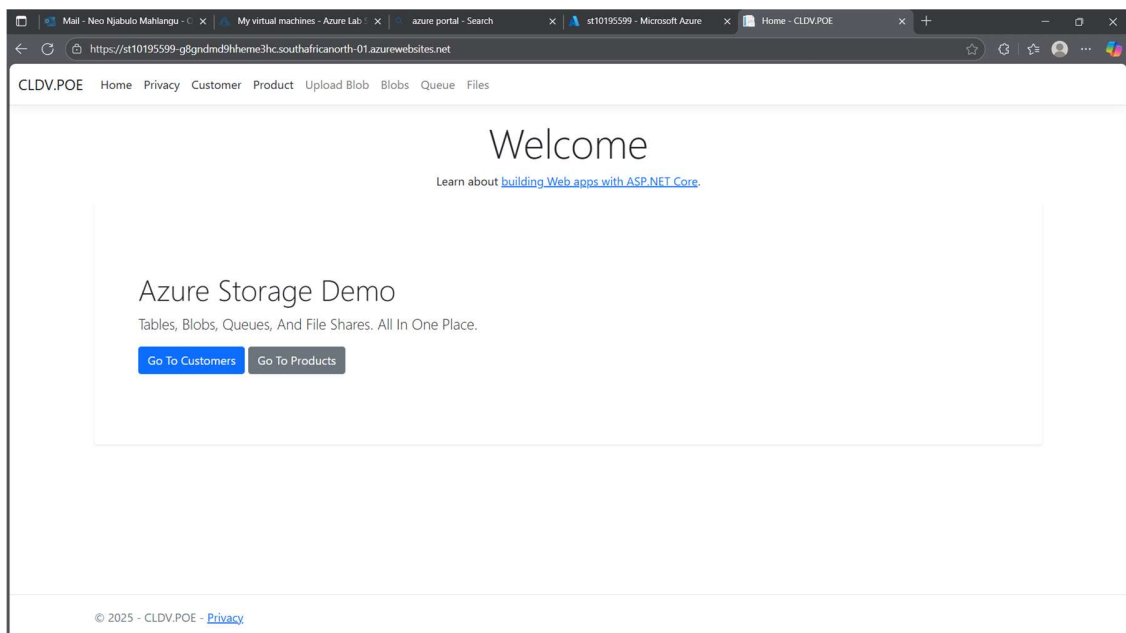
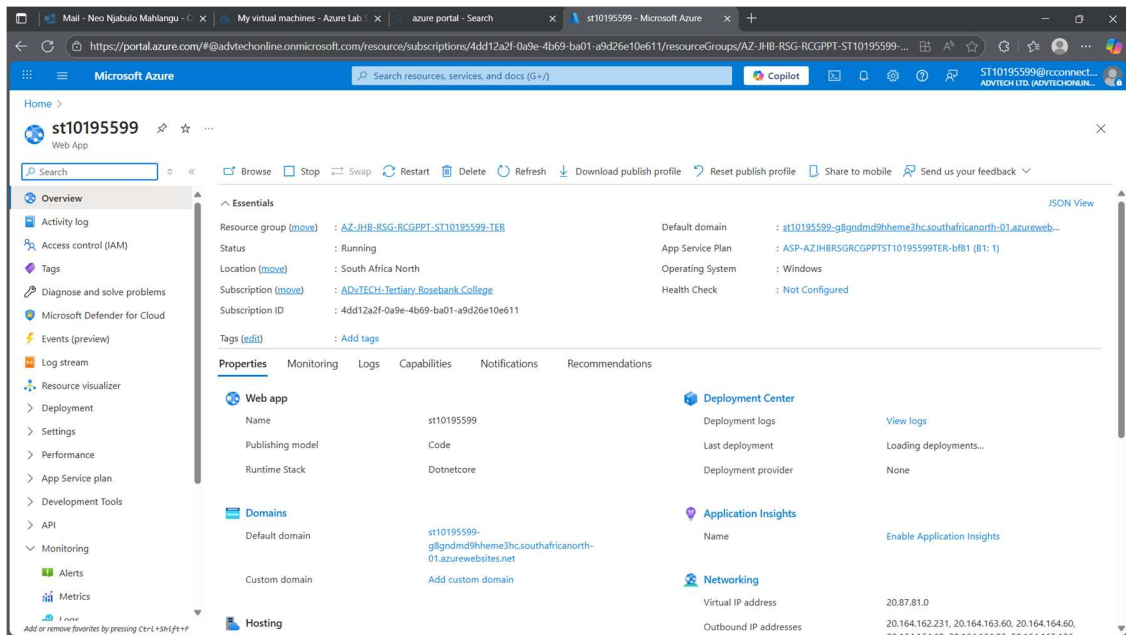
This Is The Testing For The Customers Index View On The Web App.



This Is The Testing For The Product Index View On The Web App.

5. Deployment to Azure

The application was deployed to **Azure App Service** directly from **Visual Studio 2022**. First, a publish profile was downloaded from the Azure portal and imported into the project. Using the **Publish** option in Visual Studio, the app was built and published to the configured App Service instance. Once deployed, the system was accessible via its assigned Azure web URL.



6. Challenges and Solutions

During implementation of the project, several challenges were encountered while integrating the ASP.NET Core MVC application with Microsoft Azure services. These challenges provided valuable learning opportunities and were resolved through debugging, research, and code refinements.

1. File Share Logging Error (`ArgumentOutOfRangeException`)

- **Challenge:** An exception occurred when appending logs to Azure File Share: *System.ArgumentOutOfRangeException: Specified argument was out of the range of valid values (Parameter 'length').*
This happened because an empty file range (0, 0) was uploaded when creating a new log file.
- **Solution:** The FileShareService code was updated to preallocate file size correctly and only append data when bytes were available. This prevented invalid zero-length writes and allowed logging to work as expected.

2. Incomplete CRUD Functionality for Customers

- **Challenge:** Initially, only Create, Index, and Delete were implemented for customers. This meant the project did not meet the rubric's requirement for full CRUD operations.
- **Solution:** Edit and Details actions were added to the CustomerController, along with the corresponding Razor views. Service methods (`GetCustomerAsync`, `UpdateCustomerAsync`) were also implemented in the TableStorageService to support these features.

3. Deployment to Azure

- **Challenge:** While publishing the application to Azure App Service, authentication issues were encountered with the Web Management Service, causing deployment to fail.
- **Solution:** This was resolved by downloading the correct publish profile from the Azure portal and importing it into Visual Studio. The deployment then succeeded, and the application was accessible via its Azure web URL.

7. Conclusion

This project successfully demonstrated the development of a cloud-based web application using **ASP.NET Core MVC** integrated with **Microsoft Azure services**. The system was designed to

manage both products and customers with full **CRUD functionality**, while leveraging cloud storage solutions such as **Azure Table Storage** for structured data, **Blob Storage** for images, **Queue Storage** for background messaging, and **File Share** for logging. The application was deployed to **Azure App Service**, making it accessible as a live cloud-hosted solution.

Through the implementation process, several challenges were encountered, including namespace errors, storage exceptions, and Razor syntax issues. Each problem required investigation, debugging, and adaptation of code, which strengthened problem-solving and cloud integration skills.

The key lessons learned from this project include:

- The importance of aligning namespaces and configurations to ensure smooth builds and deployments.
- How different **Azure Storage services** can be used together to create scalable and reliable applications.
- The value of proper **error handling** and debugging techniques in resolving runtime issues.
- How structured project design (Models, Views, Controllers, Services) improves maintainability and clarity.
- The process of **publishing applications to Azure App Service** and managing deployment profiles.

Overall, this project not only met the requirements of the POE but also provided practical experience in building and deploying a modern cloud-based application. It reinforced both technical knowledge and the ability to adapt when facing real-world development challenges.

REFERENCES

- Microsoft. (2025) *ASP.NET Core MVC overview*. Available at: <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview> (Accessed: 25 August 2025).
- Microsoft. (2025) *Azure Table Storage documentation*. Available at: <https://learn.microsoft.com/en-us/azure/storage/tables/table-storage-overview> (Accessed: 25 August 2025).
- Microsoft. (2025) *Azure Blob Storage documentation*. Available at: <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-overview> (Accessed: 25 August 2025).

- Microsoft. (2025) *Azure Queue Storage documentation*. Available at: <https://learn.microsoft.com/en-us/azure/storage/queues/storage-queues-introduction> (Accessed: 25 August 2025).
- Microsoft. (2025) *Azure File Share documentation*. Available at: <https://learn.microsoft.com/en-us/azure/storage/files/storage-files-introduction> (Accessed: 25 August 2025).
- Microsoft. (2025) *Deploy an ASP.NET Core app to Azure App Service*. Available at: <https://learn.microsoft.com/en-us/azure/app-service/quickstart-dotnetcore> (Accessed: 25 August 2025).
- Freeman, A. (2023) *Pro ASP.NET Core 7 MVC*. 9th edn. New York: Apress.
- Esposito, D. and Saltarello, A. (2021) *Modern Web Development with ASP.NET Core 5*. 2nd edn. Microsoft Press.